

Machine Learning Models: Neural Networks & Decision Trees

Python | TensorFlow | NumPy | From-Scratch Implementations

Timur Filimonov

Project Overview:

This project explores machine learning models implemented using both high-level frameworks (TensorFlow/Keras) and low-level NumPy implementations to better understand the mathematical foundations behind modern ML systems.

The work includes building and training neural networks, implementing evaluation metrics, and coding key decision-tree algorithms such as entropy and information gain from scratch.

Dataset

The model was trained on a dataset containing 400 input features used for multi-class classification.

Each example belongs to one of six possible categories. The dataset was used to demonstrate neural network training, classification performance, and decision tree feature splitting.

Neural Network Architecture

The neural network model was built using TensorFlow Keras Sequential API with multiple dense layers and ReLU activation.

Model Architecture:

Input Layer (2 features)

↓

Dense(120) ReLU

↓

Dense(40) ReLU

↓

Dense(6) Linear

Example implementation:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    tf.keras.Input(shape=(X_train.shape[1],)),
    Dense(120, activation="relu"),
    Dense(40, activation="relu"),
    Dense(6, activation="linear")
])
model.compile(
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01)
)
```

Training curve:

Loss vs Epoch

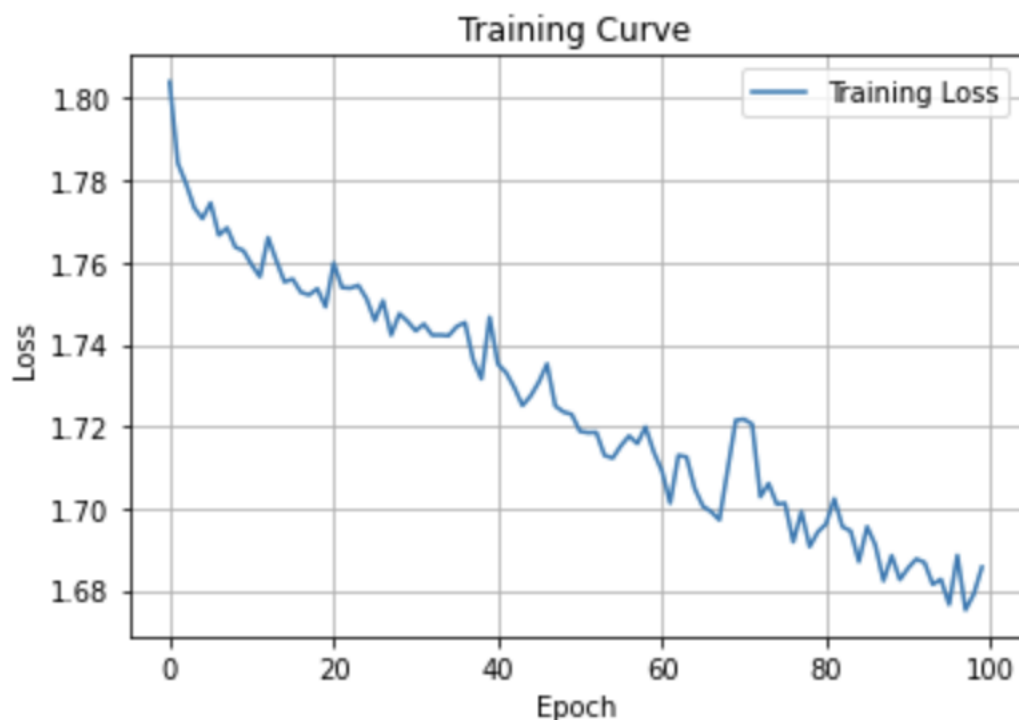


Figure 1. Training loss decreasing over epochs during neural network training using the Adam optimizer.

Regularized Neural Network

To reduce overfitting, L2 regularization was applied to hidden layers.

```
model_r = Sequential([
    tf.keras.Input(shape=(X_train.shape[1],)),
    Dense(120, activation="relu",
         kernel_regularizer=tf.keras.regularizers.l2(0.1)),
    Dense(40, activation="relu",
         kernel_regularizer=tf.keras.regularizers.l2(0.1)),
    Dense(6, activation="linear")
])
```

Regularization helps control model complexity by penalizing large weights, reducing the risk of overfitting and improving generalization during training.

Evaluation Metrics

Two evaluation metrics were implemented manually.

Mean Squared Error (MSE):

```
def eval_mse(y, yhat):
    m = len(y)
    err = 0.0

    for i in range(m):
        err += (y[i] - yhat[i])**2

    err = err / (2*m)
    return err
```

Classification Error Metric:

```
def eval_cat_err(y, yhat):
    m = len(y)
    incorrect = 0

    for i in range(m):
        if yhat[i] != y[i]:
            incorrect += 1

    return incorrect / m
```

Decision Tree Algorithms (From Scratch)

Core decision tree algorithms were implemented using NumPy.

Entropy:

```
def compute_entropy(y):  
    if len(y) == 0:  
        return 0  
    p1 = np.sum(y) / len(y)  
  
    if p1 == 0 or p1 == 1:  
        return 0  
  
    return -p1*np.log2(p1) - (1-p1)*np.log2(1-p1)
```

Dataset splitting function:

```
def split_dataset(X, node_indices, feature):  
    left_indices = []  
    right_indices = []  
  
    for i in node_indices:  
        if X[i, feature] == 1:  
            left_indices.append(i)  
        else:  
            right_indices.append(i)  
  
    return left_indices, right_indices
```

Information Gain:

```
def compute_information_gain(X, y, node_indices, feature):

    left_indices, right_indices = split_dataset(X, node_indices, feature)

    y_node = y[node_indices]
    y_left = y[left_indices]
    y_right = y[right_indices]

    H_node = compute_entropy(y_node)
    H_left = compute_entropy(y_left)
    H_right = compute_entropy(y_right)

    w_left = len(y_left)/len(y_node)
    w_right = len(y_right)/len(y_node)

    return H_node - (w_left*H_left + w_right*H_right)
```

Best Feature Selection:

```
def get_best_split(X, y, node_indices):

    num_features = X.shape[1]
    best_feature = -1
    max_info_gain = 0

    for i in range(num_features):

        info_gain = compute_information_gain(X, y, node_indices, i)

        if info_gain > max_info_gain:
            max_info_gain = info_gain
            best_feature = i

    return best_feature
```

Skills Demonstrated:

Machine Learning

TensorFlow / Keras

Neural Networks

Decision Trees

Information Theory (Entropy & Information Gain)

Model Regularization

Python / NumPy

Technologies Used:

Python

TensorFlow / Keras

NumPy

Matplotlib

Machine Learning Algorithms

Neural Networks

Decision Trees